Hazard Analysis
DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

Table 1: Revision History

Date	Developer(s)	Change
October 15	Nathan	Added Scope and Purpose of Hazard Analysis and critical assumptions
October 16	Robert	Added FMEA table
October 18	Keira	Added Safety and Security Requirements and Roadmap
October 19	Robert	Remove comments, add intro, fix typos
October 19	Keira	Added column to FMEA table to map hazards to requirements

Contents

1 Introduction **1**

2 Introduction **1**

3 Scope and Purpose of Hazard Analysis **1**

4 System Boundaries and Components **2**
4.1 Components Within the System Boundary 2
4.2 Components Outside the System Boundary 3

5 Critical Assumptions **4**

6 Failure Mode and Effect Analysis **5**

7 Safety and Security Requirements **8**
7.1 Functional Requirements 8
7.2 Non-Functional Requirements 8

8 Roadmap **9**
8.1 Requirements Planned During Capstone 9
8.2 Delayed Requirements 9

9 Appendix — Generative AI Use Disclosure **11**
9.1 AI Tools Used 11
9.2 How and Where Used 11
9.3 Verification of AI-Generated Content 11

10 Appendix — Reflection **12**

1 Introduction

2 Introduction

A hazard is a condition or event that could cause harm or loss. For DAIIM, this could include an inaccurate inference, exposure of sensitive input data or wallet keys, or mishandling a transaction involving users' funds.

Hazard analysis is especially important for DAIIM because errors cannot be easily corrected after the fact. Blockchain transactions are irreversible, so there is no central party that can intervene to reverse a harmful outcome. Identifying potential hazards early allows the team to address them while the design is still flexible.

3 Scope and Purpose of Hazard Analysis

The purpose of the hazard analysis is to identify potential hazards within the DAIIM system, examine their causes and effects, assess their associated risks, and establish mitigation strategies to eliminate or reduce the likelihood and severity of these hazards. The analysis considers the system components and boundaries defined in Section 2, with particular emphasis on the reliability of AI inference results, the integrity and security of marketplace operations, and the fairness of participant incentives. The hazard analysis will focus on identifying hazards that may lead to the following:

- **Inference accuracy:** Incorrect, inconsistent, or unreliable aggregated predictions resulting from inaccurate model outputs, weaknesses in the aggregation algorithm, or insufficiently reliable inference providers.
- **Data and transaction integrity:** Loss, corruption, or inconsistency of inference requests, prediction results, reputation information, or transaction records, potentially resulting in incorrect outcomes or payments.
- **Security and malicious participation:** Unauthorized actions or deliberate attempts to manipulate the marketplace, including collusion between inference providers to influence aggregated predictions, reputation scores, or reward distribution.
- **System availability and reliability:** Failure to submit, process, or return inference requests due to unavailable providers, failed transactions, or disruptions affecting the application's operation.
- **Economic incentives and fairness:** Incorrect payment calculations, reward distributions, or stake handling that result in financial losses for task requesters or inference providers. The analysis also considers weaknesses in the reward and penalty mechanisms that fail to incentivize accurate predictions, allow malicious or colluding providers to benefit unfairly, or undermine the reliability and economic sustainability of the marketplace.

The analysis will consider hazards arising from the interactions between the application's components, including the off-chain backend, AI inference providers, aggregation and reputation mechanisms, and on-chain programs. External dependencies, such as blockchain network infrastructure

and third-party inference services, will be considered where their failures or limitations could affect the safety, security, reliability, or intended operation of DAIIM. Hardware failures and other external events beyond the system's boundaries will otherwise remain outside the scope of the analysis.

4 System Boundaries and Components

This section divides DAIIM into its major components and identifies where the boundary of the system lies. Components inside the boundary are built, deployed, and maintained by the team, and their behaviour can be changed to mitigate hazards. Components outside the boundary are part of DAIIM's environment: DAIIM depends on them, but cannot control them, so hazards originating from them can only be detected, tolerated, or contained. The components below are based on the requirements in the SRS and the technologies listed in the Development Plan. They are preliminary and will be refined in the design documents.

4.1 Components Within the System Boundary

1. **Web Client.** The browser-based user interface, built with React, through which task requesters and inference providers use DAIIM. It connects to the user's wallet for authentication, lets users browse and create tasks, submit inference requests, register as providers, and view results, reputations, stakes, and rewards. Before each transaction, it shows the user the action being performed and the amount of tokens being transferred, and afterwards it shows the transaction's status and identifier. It also performs an initial check that request inputs conform to the task's input schema. *Related SRS requirements:* Accounts and Authorization, Inference Tasks, Inference Requests, Aggregation and Results, Transparency.

2. **On-Chain Programs.** The Solana programs, written in Rust, that hold DAIIM's authoritative state and all of its tokens. They are responsible for:

   - the registry of inference tasks and their parameters (minimum providers, reputation tolerance, acceptance period, maximum prediction time);
   - provider registration and the escrow of provider stakes;
   - escrow of requester payments and the lifecycle of each inference request (acceptance period, prediction phase, cancellation and refunds);
   - recording predictions, aggregating them into the final result, and updating provider reputations;
   - distributing rewards, and slashing or burning provider stakes.

   Because this component holds funds and blockchain transactions cannot be reversed, a defect here can cause direct and permanent financial loss. *Related SRS requirements:* Inference Tasks, Provider Registration and Staking, Provider Acceptance and Predictions, Aggregation and Results, Reputation and Rewards, Penalties, Transparency.

3. **Off-Chain Backend Service.** A Node.js and TypeScript service that performs the work that is impractical or unsafe to do on-chain. It monitors the on-chain programs for new inference requests and broadcasts them to the providers registered for the task, stores request input data and distributes it only to the requester and the providers that accepted the request,

and deletes the input data once the request is completed or cancelled. It may also submit the transactions that move a request from one phase to the next when a deadline passes, since on-chain programs cannot act on their own, and it serves marketplace information to the Web Client. Since this service is a single point that the team operates, it must not be able to change results, payments, or reputations, which are controlled only by the on-chain programs. *Related SRS requirements:* Inference Requests, Provider Acceptance and Predictions, Data Handling.

4. **Off-Chain Database.** A PostgreSQL database used by the backend service. It temporarily holds encrypted request input data, which must not be stored on the blockchain, as well as task information that is too large to store on-chain economically (such as problem descriptions, schemas, and references to training data) and indexed copies of on-chain records for fast browsing. The on-chain records remain the source of truth for any data that appears in both places. *Related SRS requirements:* Inference Tasks, Data Handling.

5. **Provider Interface.** The interface, such as an API or client library, through which an inference provider's model receives broadcast requests, accepts requests, retrieves input data, and submits predictions. It checks that predictions conform to the task's output schema before they are submitted. This component ends where the provider's own model begins. *Related SRS requirements:* Provider Acceptance and Predictions.

4.2 Components Outside the System Boundary

1. **Solana Blockchain Network.** The public Solana network, including its validators and the RPC nodes used to read state and submit transactions. DAIIM relies on it to execute the on-chain programs, store records, and provide the time used for request deadlines. It may become congested, raise fees, delay or drop transactions, or suffer outages.

2. **User Wallets.** Third-party Web3 wallets, such as browser wallet extensions, that hold users' private keys and sign transactions. DAIIM cannot recover a lost key, and cannot distinguish a compromised wallet from its legitimate owner.

3. **Inference Providers' Models and Infrastructure.** The machine learning models, computing resources, and network connections of each provider. DAIIM receives only the predictions they produce and cannot inspect the models. Providers may be unavailable, inaccurate, careless, or dishonest, and several providers may collude.

4. **Task Requesters and Their Input Data.** The people who define tasks and submit requests, and the sources of the input data they provide. DAIIM can check that inputs conform to a task's schema, but cannot verify that the data is correct, that task parameters are sensible, or that the data is not sensitive.

5. **Downstream Decisions.** The decisions that task requesters and beneficiaries, such as emergency management agencies, make using DAIIM's results. DAIIM is responsible for providing inferences and information about their quality, but not for how the results are used.

6. **Hosting Infrastructure and Browsers.** The cloud or server infrastructure that hosts the backend service and database, and the web browsers on which the Web Client runs.

5 Critical Assumptions

The following assumptions describe the conditions under which the DAIIM system is expected to operate. These assumptions establish the operating context of the system and identify external conditions on which its functionality depends. Potential failures and malicious behaviour within the system, such as inaccurate predictions, provider collusion, and manipulation of reward distribution, are not assumed to be impossible and will instead be addressed through the hazard analysis.

- The underlying blockchain network is operational and capable of processing transactions under normal operating conditions.
  - This assumption establishes the expected operating environment for on-chain marketplace operations. Temporary network outages, congestion, and failed transactions remain potential hazards that must be considered.
- Inference providers have access to the computational resources and model implementations required to execute the inference tasks they accept.
  - This assumption defines the expected capabilities of participating providers. Provider unavailability, insufficient computational resources, and failed inference requests remain potential hazards.
- A minimum number of two inference providers is available during the initial operation of the marketplace.
  - This assumption establishes that the marketplace has some initial provider participation to process inference requests and evaluate the aggregation, reputation, and reward mechanisms.
- Users have access to a modern web browser and an internet connection to interact with the application.
  - This assumption defines the minimum environment required to access the user interface. Network interruptions and browser compatibility issues remain potential causes of service disruption.

6 Failure Mode and Effect Analysis

Table 2: Failure Modes and Effects Analysis

Failure	Ref.	Failure Modes	Failure Effect	Failure Causes	Recommended Actions	Safety Reqs.
A required on-chain transaction does not succeed	H1.1	RPC endpoint is unavailable	The intended on-chain state change or token transfer is delayed or not recorded. Depending on the operation, an inference request may not be submitted or broadcast, a prediction may not be recorded, a reputation update may be missed, or a token transfer or burn may fail. These failures could result in direct financial losses from missed or incorrect token transfers, indirect financial losses if outdated reputation scores affect provider rewards, or a suboptimal final output due to incorrect provider reputations.	RPC outage, network congestion, or an unavailable endpoint	Check transaction status and RPC health. Retry only after confirming that the transaction was not processed, and show the user its current status.	FR.SA.1
	H1.2	The account lacks sufficient funds		Insufficient token balance for the transfer or insufficient coin (native-tokens) balance for transaction fees	Check involved account balances and explain which balance is insufficient and the required amount; allow the user to add funds and retry.	FR.SA.2, FR.SA.3
	H1.3	Wallet authorization is missing or invalid		The user rejects the signature request or the submitted signature is invalid	Explain that authorization is required and allow the user to initiate another signature request.	FR.SA.4
	H1.4	The transaction expires before confirmation		Network or processing latency causes the transaction's validity period to expire	Check whether the transaction was processed; if not, regenerate the transaction and resend it.	FR.SA.5
	H1.5	Transaction validation or execution failure.		Missing or invalid signatures, invalid instruction data or account state, or unmet program constraints, such as insufficient permissions or an account not being authorized to perform the requested operation.	Display the failure reason when available and prompt the user to resolve the issue (such as authorizing a different account) before retrying the transaction.	FR.SA.6, NFR.SA.1
A provider's private key is compromised	H2.1	The private key is exposed through logs, error messages, or insecure storage	An attacker may control the provider's wallet, impersonate the provider, access provider data, or transfer or forfeit the provider's funds and stake.	The client library or provider software logs secrets, stores them insecurely, or exposes them to other local processes	Keep private keys under the provider's wallet control; do not send them to or store them in the client library. Redact secrets from logs and protect any required local credentials.	NFR.SA.2
	H2.2	The private key is intercepted during transmission		Unencrypted transport, incorrect TLS configuration, or disabled certificate validation	Use correctly configured TLS for all communications; validate certificates and never transmit raw private keys through the API.	NFR.SA.3
	H2.3	A vulnerable or compromised client-library dependency exposes the private key		A security flaw in a third-party software dependency allows an attacker to access or steal private keys used by the library.	Minimize the number of third-party dependencies, use audited and trusted package versions, and promptly update dependencies when security vulnerabilities are identified.	NFR.SA.4

Continued on next page

Table 2 – continued from previous page

Failure	Ref.	Failure Mode	Failure Effect	Failure Cause	Recommended Action	Safety Reqs.
A financial operation is handled incorrectly, resulting in an incorrect allocation or transfer of tokens.	H3.1	The task requester is charged an incorrect amount	The task requester is charged more or less tokens than intended, causing direct financial losses for the task requester or task participants.	Incorrect financial calculation	Validate payment calculations and test the on-chain financial instructions against expected amounts. Before submitting a stake transaction, simulate it using available tools and verify that the expected balance changes match the actual change.	FR.SA.7, FR.SA.8
	H3.2	Provider rewards are calculated or distributed incorrectly	Providers are underpaid, overpaid, or receive duplicate rewards.	Incorrect reward calculations, allocation logic, or duplicate processing	Validate reward calculations and ensure each request distributes rewards only once.	FR.SA.9
	H3.3	Staked tokens are returned or burned under the wrong conditions	Providers may lose their stake incorrectly or retain tokens that should have been burned.	Incorrect handling of voluntary unregistration or forced removal of inference providers	Validate stake return and burn instructions with tests for each removal scenario.	FR.SA.10
The final inference result is not reasonably close to the ground truth.	H4.1	The aggregation algorithm or reward incentives fail to favor accurate predictions	Users receive an inaccurate final inference result and may make decisions based on incorrect information. Inaccurate results may also undermine trust in the marketplace and its economic sustainability.	The aggregation method handles provider disagreement or outlying predictions poorly, or token rewards and penalties do not align provider incentives with prediction accuracy	Evaluate aggregation on tasks with known ground truth and varied provider prediction quality (see POC plan). Compare the simulated net rewards of honest providers producing low-error predictions with malicious providers.	SA.HA.1
	H4.2	Malicious or low-quality providers significantly influence the aggregated result		The aggregation and reputation mechanisms do not sufficiently reduce the influence of inaccurate or dishonest predictions	Simulate different proportions of honest and malicious providers. Compare aggregated results against known ground truth and report the observed error. Use the results to identify and communicate the provider composition thresholds under which the system is expected to meet its accuracy criteria.	–
	H4.3	Colluding providers submit similar inaccurate predictions to manipulate the aggregation		A coordinated group constitutes a large enough portion of the network to skew aggregation, reputation scores, or rewards, particularly when ground truth is unavailable	Simulate colluding groups of different sizes and measure their effect on result accuracy, reputation, and rewards. Document the provider proportions at which collusion can materially influence results.	SA.HA.2

Continued on next page

Table 2 – continued from previous page

Failure	Ref.	Failure Mode	Failure Effect	Failure Cause	Recommended Action	Safety Reqs.
An inference request does not receive enough predictions.	H5.1	Too few providers accept the request (fewer than the task's minimum)	The requester receives no result, or an unreliable one based on too few predictions. The requester's payment is returned and the inference request ends.	Providers are unavailable or unwilling to participate and do not accept the request before the end of the acceptance period	Cancel the request and refund the requester in full.	FR23
	H5.2	Providers accept the request, but fewer than the minimum number submit a prediction		Accepted providers are unavailable, fail, or do not submit before the end of the prediction phase	Cancel the request and refund the requester in full. Slash the stake of each provider that accepted but did not submit, and decrease their reputation.	FR52, FR30

7 Safety and Security Requirements

Based on the hazard analysis above, the following safety and security requirements have been outlined. Each requirement addresses a hazard in the FMEA table.

7.1 Functional Requirements

FR.SA.1: The system shall not resubmit a transaction until it has confirmed that the original transaction was not processed and shall maintain logs of all transaction statuses and log verified transactions publicly on-chain.

FR.SA.2: The system shall verify if the wallet of a user has sufficient funds before requesting a signature and logging the transaction.

FR.SA.3: The on-chain programs shall reject any transaction that the wallet cannot cover at the time of execution, regardless of any balance check made earlier.

FR.SA.4: The system shall not submit any transaction which is not signed by an authorized wallet on-chain or off-chain.

FR.SA.5: The system shall wait for validity period to end and status to be checked before logging transaction as failed and attempting to reprocess.

FR.SA.6: When a transaction fails, the system shall send the error to the user and allow them to retry.

FR.SA.7: Before a transaction is submitted for signing, the system shall simulate it and verify that the resulting balance changes equal the amounts displayed to the user, and shall not submit it otherwise.

FR.SA.8: The amount charged to a requester for an inference request shall equal the payment amount the requester authorized.

FR.SA.9: The system shall validate each reward calculation before distributing it, and shall not distribute rewards whose total exceeds the reward for the request.

FR.SA.10: The system shall confirm provider status before distributing rewards/slashing tokens.

SA.HA.1 : Providers shall be given random tests with a known ground truth to identify malicious providers.

SA.HA.2 : The system shall send invitation to accept an inference request to providers randomly, to reduce the possibility of premeditated collusion attacks.

7.2 Non-Functional Requirements

NFR.SA.1: The system shall send readable error messages to users within 2 seconds of a detected failure.

NFR.SA.2: All signing shall happen within the user's wallet and private information shall not be logged on-chain.

NFR.SA.3: All communication between DAIIM components, and between users and DAIIM, shall use TLS with certificate validation enabled to ensure data protection.

NFR.SA.4: The system shall scan third-party dependencies for known security vulnerabilities, verified through testing.

Other mitigation strategies include the implementation of simulations to gather data and justify protocols for mitigating malicous users and collusion attacks.

8 Roadmap

8.1 Requirements Planned During Capstone

The requirements that can be implemented within the timeline of the project are listed below, along with justification.

- FR.SA.1: Duplicate transactions are the most direct way users could lose funds. Checking a transaction's status before retrying is a small amount of logic, and we will need it as soon as we start testing transactions on Devnet. Transaction statuses are also needed to show progress to users and to debug the on-chain programs, and confirmed transactions are already recorded on-chain as part of the core design.
- FR.SA.2: Checking balances before asking for a signature avoids failed transactions that could have been prevented. It is a simple lookup and is needed for the transaction preview anyway.
- FR.SA.3: Enforcing this in the on-chain programs protects funds no matter what the client checks, so it belongs with the development of the programs themselves.
- FR.SA.4: The blockchain rejects unsigned transactions, so making sure our client never submits one is part of basic transaction handling.
- FR.SA.5: Transactions will expire during network congestion on Devnet. Retry handling needs to be built early so that expired transactions are not submitted twice.
- FR.SA.6: Users must be told when a transaction fails and be able to retry. The same error handling will also be what we rely on for debugging during development.
- FR.SA.7: Wrong payment amounts are likely to appear while we develop the payment instructions. Simulating transactions before signing will catch them before users are asked to authorize anything.
- FR.SA.8: Charging the amount the requester authorized is the core of financial correctness, and a mistake here causes direct losses. It is part of the payment logic in the on-chain programs.
- FR.SA.9: A total-reward check is a simple invariant in the reward logic, and it catches calculation errors before tokens move.
- FR.SA.10: Paying or slashing a provider who is no longer registered would break the stake rules. Checking the provider's status first is a small addition to the staking logic.
- NFR.SA.1: Readable errors matter for both usability and debugging. The time limit is measured inside our own client, so we can test it on Devnet.
- NFR.SA.2: Signing happens in the user's wallet by design (the embedded wallet for requesters, the provider's own signer through the SDK), so this requirement needs very little extra development.
- NFR.SA.3: TLS with certificate validation is a hosting and library configuration, and it is needed for any deployment, including demos.
- NFR.SA.4: The use of the continuous integration pipeline to scan third-party dependencies will aid in identifying vulnerabilities promptly and is a natural addition when incorporating third party dependencies into the system. Once implemented, the scans will occur automatically.

8.2 Delayed Requirements

The requirements that will be implemented in the future and the justification are detailed below.

- SA.HA.1: Giving providers random tests with known answers needs tasks with known ground truth, test requests that cannot be told apart from real ones (otherwise providers would only behave well on tests), and a way to pay providers for requests that have no requester. This is an additional feature that would enhance

security and improve provider integrity, but is beyond the scope of the project and is not essential to the main functionality.

- SA.HA.2: Randomly inviting providers requires a secure, verifiable source of randomness on Solana, and conflicts with the current design in which every registered provider receives each request. It would also require reworking how the minimum number of providers is met.

9 Appendix — Generative AI Use Disclosure

This section provides an overview of how generative AI tools were used in the making of this document.

9.1 AI Tools Used

- Claude Opus 5.5 (Anthropic), used through Claude Code.

9.2 How and Where Used

Claude Opus 5.5 was used throughout this document to help ideate and draft its content. The team provided the tool with the SRS and the Development Plan, and used it to brainstorm the hazards, components, assumptions, and requirements that the document should address, and to produce first drafts of the text. In Section 3 (System Boundaries and Components), for example, it proposed a division of DAIIM into components inside and outside the system boundary, drafted the description of each component, and linked each component to the groups of functional requirements in the SRS that it is responsible for. It was also used to draft the text of this appendix.

The tool was used as an aid, not as a replacement for the team's own work. Team members were involved in every sentence of the document: each AI-assisted draft was read, discussed, and edited by the team, and ideas that did not fit the project were discarded or rewritten.

9.3 Verification of AI-Generated Content

Every sentence in this document was reviewed by at least one team member before it was included. AI-assisted content was checked against the SRS and the Development Plan to confirm that it describes responsibilities and behaviour that the requirements actually assign to DAIIM, that it does not introduce requirements or design decisions the team has not agreed to, and that it is consistent with the technologies the Development Plan lists. References to SRS requirements were checked by hand. Content that is still undecided, such as how request phases are advanced when deadlines pass, is marked as preliminary and will be confirmed in the design documents. The team remains responsible for the accuracy, completeness, and consistency of this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

10 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable? One thing that went well during this deliverable was the concretization of the project that came out of defining functional requirements. Up until this point, the problem statement, goals and development plan gave us a good definition of the scope and the reason for the project, but there have been several looming questions about what actions the system will perform. I think we were able to successfully define the core functional requirements for the system, at a good level of abstraction. Our requirements are specific enough to say a lot about what the system is responsible for, while not proposing possible designs. This process allowed us to answer a lot of the questions we had discovered during earlier deliverables. Although these requirements are likely to evolve as the project continues (especially during the POC), this felt like a significant milestone and marker of progress.

2. What pain points did you experience during this deliverable, and how did you resolve them? A pain point we experienced for this deliverable is the organizational overhead that arose because of the volume of written content required. Because the SRS document and Hazard Analysis are large documents, it was often not straightforward to allocate sections for parallel development, review each other's work, and keep track of the work. We addressed these problems by assigning responsibility for each section, providing constructive feedback in GitHub pull requests, and reviewing the integrated documents together before final submission. This made it easier to work in parallel while catching gaps, overlap, and inconsistencies during review. It also made it a lot easier for each member to keep track of the work they were responsible for. We also plan to treat the documents as evolving artifacts, revisiting and refining earlier sections when later decisions clarified the system's requirements and hazards. We also established a standard of creating a GitHub issue for every major piece of work and tracking its progress on the Kanban board.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about? We had already considered inaccurate or unreliable predictions, provider unavailability, and malicious participation in our dev plan (POC plan). These are the main risks that the POC aims to prove are avoidable. The Hazard Analysis brought more risks into focus relating to the software environment, including failed or expired transactions, private-key exposure through software or dependencies, and incorrect reward or stake handling. These emerged after defining the main functional requirements and considering how each responsibility could fail.

4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider? We talked about two other types of risk other than physical harm in the hazard analysis. Privacy risks arise if sensitive information is exposed to unauthorized parties. This can harm users and undermine their trust in the system. The other types of risk are legal and financial risks. Legal and financial risks can arise when transactions fail, are duplicated, or charge an incorrect amount. Users could lose funds or face disputes, and the project could incur

liability. These risks are important to consider because they directly affect our stakeholders and, if we cannot mitigate them, could threaten the project's success.